

Model Predictive Control of a Differential-Drive Robot

Edward Sandra, Mathias Bos, Wilm Decré

1 Introduction

The Alvik is a small differential-drive robot equipped with a five-zone forward-facing time-of-flight sensor. In this project the robot itself performs no decision-making: it reports its pose and range measurements and executes whatever velocity command it receives. All control decisions are instead computed externally by a Model Predictive Controller (MPC), which at each control cycle (10 Hz) uses the current state, the goal, and the locally perceived obstacles to solve a finite-horizon optimal control problem and applies the first element of the resulting control sequence.

In this assignment, we will build such an MPC controller for the Alvik: given its current pose, a target goal, and the obstacles detected by its sensor, the controller must compute a sequence of velocity commands that drives the robot to the goal while avoiding collisions. The controller is formulated and implemented using [Impact](#), a Python interface for optimal control problems built on CasADi. The supporting infrastructure, perception, simulation, plotting, and solver warm-starting, is provided. The task consists of completing the function `build_mpc()` in `mpc.py`.

2 Environment Setup

A Python environment with `casadi`, `impact-meco`, `numpy`, and `matplotlib` is required. Either run `pip install impact-meco matplotlib` in a virtual environment, or use the provided Docker image. The latter is recommended for participants who are unfamiliar with Python virtual environments or who wish to avoid installing Python and its dependencies on their host machine.

3 Provided Code Base

File	Description
<code>mpc.py</code>	The file to be completed. All other files are support code.
<code>geometry.py</code>	Physical and tuning constants: sensor geometry, robot limits, margins.
<code>perception.py</code>	Converts raw range measurements into fitted circular obstacles.
<code>plant.py</code>	Ground-truth simulated robot: unicycle dynamics with a ray-cast sensor model.
<code>nav_core.py</code>	Combines perception and the MPC into a single decision per control cycle.
<code>obstacles.py</code>	Parses obstacle specifications of the form " <code>cx,cy,r cx,cy,r ...</code> ".
<code>run_mpc.py</code>	Evaluation script. Runs the controller without robot or network.

Only `mpc.py` needs to be modified. The remaining files may be consulted for context but should not require changes.

4 System Model

The robot is modelled as a unicycle: a point mass with world-frame pose (p_x, p_y, θ) , where p_x, p_y are given in meters and θ in radians, actuated by a forward speed v (in m/s) and an angular

rate ω (in rad/s):

$$\dot{p}_x = v \cos \theta \quad (1)$$

$$\dot{p}_y = v \sin \theta \quad (2)$$

$$\dot{\theta} = \omega \quad (3)$$

Reverse motion is disallowed ($v \geq 0$); a goal located behind the robot must therefore be approached by turning around. Angular velocities can be positive (counterclockwise) or negative (clockwise).

5 Implementation Task

Open `mpc.py` and locate `build_mpc()`. The state variables (`px`, `py`, `theta`), control variables (`v`, `omega`), and all run-time parameters (`x_current`, `goal`, `obs`, `v_max`, `omega_max`) are already declared. Five components of the problem formulation are missing, each marked with a comment and a `raise NotImplementedError(...)` placeholder that must be replaced with a working implementation. The five components depend on one another and must be completed in order: each placeholder raises an exception until the preceding one has been implemented, so at any point in the process exactly one next step is indicated.

Task 1. System dynamics

Specify the state derivatives with `mpc.set_der(state, expr)`, once for each of `px`, `py`, and `theta`, corresponding to Equations (1)–(3).

Task 2. Initial condition

The optimization must be constrained to start from the currently measured pose. Impose this with `mpc.subject_to(mpc.at_t0(state) == x_current[i])` for each of the three states, where `x_current[0]`, `[1]`, and `[2]` correspond to p_x , p_y , and θ , respectively.

Task 3. Control bounds

Impose the following bounds using `mpc.subject_to(...)`:

- $v \in [0, v_max]$
- $\omega \in [-omega_max, omega_max]$

The bounds must reference the parameters `v_max` and `omega_max`, not the module-level constants `V_MAX` and `OMEGA_MAX`. The parameters can be updated at run time (for instance from an operator interface); the constants merely supply their initial values. Bounding against `V_MAX` and `OMEGA_MAX` would cause any subsequent change to the limits to be silently ignored.

Task 4. Obstacle avoidance

Each obstacle corresponds to a column of the parameter `obs`: `obs[0,j]`, `[1,j]`, and `[2,j]` give its centre (c_x, c_y) and radius r , respectively, for $j = 0, \dots, n_obs - 1$. The robot is required to remain outside a keep-out disk of radius $r + \text{MARGIN}$ around every obstacle. An unused obstacle slot is parked far from the robot with $r = 0$; the chosen formulation should render such a slot harmless without being treated as a special case.

The means by which this requirement is enforced is left to the workshop participant. Suitable approaches include a hard constraint on the distance to each obstacle, a penalty term added to the objective (Task 5), or a combination of the two; each involves a different trade-off between guaranteed clearance and numerical robustness, and either choice is acceptable provided the resulting controller satisfies the criteria of Section 8.

Task 5. Objective function

Formulate the cost functional. A stage cost is incurred at every point along the horizon and summed with `mpc.sum(...)`; a terminal cost is incurred once, evaluated at the final predicted state with `mpc.at_tf(...)`. Both terms are added to the problem with `mpc.add_objective(...)`, called once per term. If the obstacle constraint from Task 4 was implemented as a penalty, it should be included in the stage cost below.

$$L_{\text{stage}} = W_POS \cdot [(p_x - g_x)^2 + (p_y - g_y)^2] + W_V \cdot v^2 + W_OMEGA \cdot \omega^2 \\ + W_HEAD \cdot \text{head_gate} \cdot \text{head_align} + \text{obs_penalty} \quad (\text{if applicable}) \quad (4)$$

$$L_{\text{term}} = W_TERM \cdot [(p_x - g_x)^2 + (p_y - g_y)^2] \Big|_{t=t_f} \quad (5)$$

The heading-alignment term (`head_align`, `head_gate`) is given, and penalizes the angle between the robot's heading and the direction to the goal. It is required because, with reverse motion disallowed, the position cost alone has zero gradient at $v = 0$ whenever the goal lies behind the robot, giving the optimizer no incentive to initiate a turn. The term is gated to zero near the goal so that it does not interfere with the terminal cost once the robot has effectively arrived.

6 Verification

Once all five components have been implemented, three checks are available. They should be run from the directory containing `mpc.py`.

```
python mpc.py --benchmark
```

Constructs the MPC and solves it over 200 consecutive cycles along a representative approach to a two-obstacle scene, reporting solve-time statistics. Since the control loop operates at 10 Hz, each solve has a 100 ms budget; a mean or 95th-percentile solve time approaching this budget indicates a problem, typically a hard obstacle constraint combined with a poor initial guess. A correctly functioning solve should require only a few milliseconds.

```
python mpc.py --plot
```

Solves a single scene and saves `mpc_solution.png`, showing the planned path around one obstacle and the predicted heading over the horizon. This provides a quick check of a single solve without exercising the full control loop.

```
python run_mpc.py
```

The primary evaluation: a simulated robot is driven toward a goal, with the loop closed at 10 Hz by the implemented controller and the result displayed as a live animation. An alternative obstacle layout and goal may be specified as follows.

```
python run_mpc.py --obstacles "0.7,0.3,0.14 1.6,0.05,0.12" --goal
1.5,0
```

The flags `--no-gui --max-steps 300` produce a printed trajectory instead of a plot window, which is useful over a remote connection or for an automated check.

7 Hardware Deployment

Deployment on the physical Alvik robot is not self-service. Participants who wish to have their controller evaluated on the robot must submit their completed `mpc.py` file through the following form:

<https://tinyurl.com/ccta-alvik>

Submitted files are flashed onto the robot by the course staff. A submission should only be made once the checks described in Section 6 pass, since the same file is used for the hardware run.

8 Evaluation Criteria

A correct implementation is expected to satisfy the following.

- `mpc.py --benchmark` completes without error and reports a mean solve time of a few milliseconds, well within the 100 ms budget.
- `run_mpc.py` drives the robot from the initial pose to the goal along a smooth trajectory that avoids every obstacle in its path.
- The reported `status` equals 0 (success) for essentially every cycle. An occasional non-zero status in close proximity to an obstacle is not necessarily indicative of an error, particularly for a hard-constraint formulation, but it should not persist over successive cycles.

9 Common Implementation Errors

- **Bounding against the wrong symbol.** The controls must be bounded using the parameters `v_max` and `omega_max`, not the constants `V_MAX` and `OMEGA_MAX` (Task 3).
- **Sign error in the obstacle constraint.** Whatever quantity is used to express the keep-out condition, it must be defined consistently with the direction of the inequality or penalty applied to it. A robot that drives through obstacles suggests an inverted sign; a robot that treats free space as forbidden suggests the same error in the opposite direction.
- **Missing clipping in a penalty-based approach.** If obstacle avoidance is implemented as a penalty term, it should vanish outside the keep-out disk, typically via `ca.fmax(..., 0.0)`. Without this clipping, positions outside an obstacle contribute a negative penalty, which the optimizer can exploit by remaining close to obstacles rather than avoiding them.
- **Use of Python control flow or built-ins on symbolic expressions.** Any expression within `build_mpc()` involving `px`, `py`, `theta`, `v`, `omega`, or a parameter is a CasADi symbolic expression rather than a numerical value. The corresponding CasADi functions (`ca.fmax`, `ca.sqrt`, etc.) must be used in place of Python's built-in equivalents.
- **A robot that fails to move.** If the robot remains stationary, verify that the heading term was included in the objective (Task 5); its omission can leave the optimizer with no incentive to leave $v = 0$ when the goal lies behind the robot.